

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА ПОДПОМАГАНЕ НА
РЕШЕНИЯТА ПРИ ИЗБОР И ПОДРЕЖДАНЕ
НА ПОРТФЕЙЛ ОТ ИТ ПРОЕКТИ ПРИ
БЮДЖЕТНИ И РЕСУРСНИ ОГРАНИЧЕНИЯ**

КУРСОВ ПРОЕКТ

по дисциплина „Информационни технологии за бизнес мениджмънт“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Иван Станков и Проф. д-р Димитър Вергиев

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Информационни технологии и подпомагане на управленските решения	3
1.2. Управление и моделиране на бизнес процеси	3
1.3. Многокритериален анализ на решенията	4
1.4. Избор на портфейл и управление на инвестиционния риск	4
1.5. Изводи от прегледа	5
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Метод за многокритериална оценка	6
2.2. Метод за решаване на задачата за подбор	6
2.3. Технологична основа	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	9
3.1. Слоеви и отговорности	9
3.2. Модел на данните	10
3.3. Формат на сценария и на резултата	12
3.4. Анализ на риска	12
3.5. Анализ на чувствителността	12
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	14
4.1. Обща структура на изходния текст	14
4.2. Слой на модела	14
4.3. Слой на оценяването	15
4.4. Слой на подбора	16
4.5. Слой на риска	17
4.6. Слой на представянето	18
4.7. Тестове	18
V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА	20
5.1. Какво се проверява	20

5.2. Среда за измерване	20
5.3. Набор от сценарии	21
5.4. Процедура	21
5.5. Форма на резултатите	22
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	23
6.1. Коректност върху задачи с известно решение	23
6.2. Съпоставка на двата метода за многокритериална оценка	23
6.3. Избраният портфейл и критичният път на програмата	24
6.4. Точно решаване срещу лакома евристика	25
6.5. Сходимост на метода на Монте Карло	26
6.6. Разпределение на стойността и на срока	27
6.7. Устойчивост на препоръката	28
6.8. Обобщение на измереното	29
ЗАКЛЮЧЕНИЕ	30
ЛИТЕРАТУРА	32
ПРИЛОЖЕНИЯ	34
Приложение А. Схема на сценария	34
Приложение Б. Изход на системата	36
Приложение В. Изходен текст	39
Приложение Г. Пълни резултати	42

УВОД

Всяка организация с информационни технологии има повече идеи за проекти, отколкото пари и хора да ги изпълни. Изборът кои от тях да се финансират и в какъв ред е управленско решение, но то се взема върху количествени данни: разчетна стойност, продължителност, зависимости между дейностите, очаквана полза и риск. На практика решението често се взема в електронна таблица, чиито допускания не са записани никъде, а резултатът не може да бъде възпроизведен след промяна на бюджета. Такъв процес не е одитируем и не понася въпроса „какво се променя, ако бюджетът падне с една пета“.

Настоящият курсов проект разработва програмна система, наречена **Quorum**, която подпомага това решение вместо да го замества. Системата приема описание на бизнес процеса като насочен граф от дейности със стойности, продължителности и зависимости, оценява проектите кандидати по няколко критерия едновременно и избира портфейл, който максимизира сумарната полза при зададени ограничения по бюджет и капацитет. Към резултата се прилага анализ на чувствителността, който показва как се променя препоръката, когато допусканията се движат.

Целта на проекта е да покаже как класически методи на управленската наука, многокритериален анализ и целочислена оптимизация, се вграждат в работеща информационна система за подпомагане на решенията и как получената препоръка става проследима до входните допускания.

За постигането на целта са формулирани следните **задачи**:

1. преглед на предметната област: информационни технологии в бизнеса, управление и моделиране на бизнес процеси, системи за подпомагане на решенията и управление на инвестиционния риск;
2. анализ на възможните решения по два независими избора, метод за многокритериална оценка и метод за решаване на задачата за избор на портфейл, и обосновка на направения избор;
3. проектиране на системата: модел на данните, слоеве, интерфейси между тях;
4. програмна реализация на ядрото, оптимизационния слой и потребителския интерфейс за въвеждане на сценарии;
5. методика за експериментална проверка върху сценарии с известни свойства;

6. провеждане на експериментите, анализ на резултатите и на чувствителността на препоръката.

Обхватът на проекта е решението за избор и подреждане на портфейла. Извън обхвата остават проследяването на изпълнението след одобрението на портфейла, счетоводното отчитане и интеграцията с работеща система за планиране на ресурсите на предприятието. Начините, по които такава интеграция би се осъществила, са обсъдени, но не са реализирани.

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката на експерименталната проверка, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Информационни технологии и подпомагане на управленските решения

Разделението на управленските решения на структурирани, полуструктурирани и неструктурирани е въведено от Gorru и Scott Morton [1] и остава полезно и днес, защото определя каква роля може да поеме една информационна система. Структурираното решение се автоматизира изцяло. Неструктурираното остава изцяло на човека. Полуструктурираното, а изборът на портфейл от проекти е точно такова, се поддава на изчислителна поддръжка само частично: изчислима е следствената част, а преценката кои критерии колко тежат остава управленска. Power [2] систематизира класовете системи за подпомагане на решенията и подчертава същото: системата предоставя модел и данни, но не поема отговорността за избора.

Оттук следва проектното изискване, което определя целия облик на разработваната система. Тя не произвежда „правилния отговор“, а прави препоръката проследима: за всяка избрана позиция трябва да е видимо кое ограничение я е допуснало и коя оценка я е класирала. Arnott и Pervan [3] стигат до същото изискване по друг път: при преглед на изследванията в областта те отчитат, че голяма част от предлаганите системи не са проверими от потребителя си, защото допусканията им не са изнесени навън, и посочват това като причина за слабото им приемане на практика. Проследимостта на препоръката следователно не е удобство, а условие системата изобщо да бъде използвана.

1.2. Управление и моделиране на бизнес процеси

Съвременното управление на бизнес процеси се разглежда като жизнен цикъл от идентификация, откриване, анализ, преработване, внедряване и наблюдение [4]. За настоящия проект е съществена фазата на анализ, при която процесът е вече описан формално и подлежи на количествена оценка по време, стойност и ресурс. Нотацията BPMN 2.0 [5] е приетият стандарт за такова описание и дава семантика на потока от дейности, на разклоненията и на събитията.

Мрежовият модел на дейностите с продължителности и зависимости, върху който

стъпва изчисляването на най-ранните и най-късните срокове, е по-стар от BPMN и е известен като метод на критичния път. Той дава на системата две неща наведнъж: най-ранния възможен срок за завършване на един проект и запаса от време на всяка негова дейност. Запасът е важен, защото дейност без запас превръща всяко закъснение в закъснение на целия проект, а това е вход към оценката на риска. Методът е описан от Kelley и Walker [6] през 1959 година върху детерминирани продължителности. Успоредно с него и през същата година е предложен PERT [7], който заменя единичната продължителност с тройна оценка, оптимистична, най-вероятна и песимистична, и извежда от нея очаквана стойност и дисперсия. Двата подхода не се изключват: изчисляването на сроковете е едно и също, а тройната оценка добавя към резултата разсейване, което детерминираният вариант няма. Настоящата разработка използва точно това съчетание.

1.3. Многокритериален анализ на решенията

Проектите кандидати не се сравняват по един показател. Стратегическата стойност, техническият риск, регулаторната необходимост и очакваната възвръщаемост са несъизмерими и част от тях се измерват само по субективна скала. Методът на аналитичната йерархия [8, 9] решава това чрез попарни сравнения и извежда теглата на критериите от собствен вектор на матрицата на сравненията, като предлага и мярка за съгласуваност на преценките. Методът TOPSIS [10] подхожда различно: класира алтернативите по относителна близост до идеалното решение и отдалеченост от най-лошото, при предварително зададени тегла. Triantaphyllou [11] сравнява семейството методи и показва, че различните методи върху едни и същи данни невинаги дават едно и също класиране. Именно това несъвпадение е причината двата метода да бъдат реализирани и съпоставени в Раздел II, вместо единият да бъде избран предварително.

1.4. Избор на портфейл и управление на инвестиционния риск

Идеята, че портфейлът се оценява като цяло, а не като сбор от независимо добри позиции, произхожда от Markowitz [12]. Пренесена върху портфейл от ИТ проекти, тя означава, че сумарната полза не е единственият критерий: концентрацията на риск в няколко зависими проекта е свойство на съвкупността. Стандартът за управление на портфейли [13] описва управленската рамка около това решение, включително

балансирането на портфейла спрямо стратегическите цели. Archer и Ghasemzadeh [14] предлагат рамка, в която изборът е разделен на последователни етапи, предварителен подбор, индивидуална оценка и избор на съвкупността, и подчертават, че смесването на етапите е чест източник на непроследими решения. Разделението на слоеве в Раздел III следва същата логика.

От изчислителна страна задачата за избор на подмножество проекти с максимална сумарна полза при ограничен бюджет е разновидност на задачата за раницата. Класификацията на вариантите и точните алгоритми са изложени в [15] и [16]. Задачата е NP-трудна в общия си вид, но практическите размери в разглеждания случай са малки, десетки проекти, а не милиони, което прави точното решаване напълно приложимо. Добавянето на ограничение по капацитет за повече от един ресурс превръща формулировката в многомерна задача за раницата, чийто преглед е даден от Fréville [17]; там е изложено и защо линейната релаксация остава използвана като горна граница и в многомерния случай, което е пряко използвано в Раздел IV. Отношенията на предхождане между проектите се добавят като отделно семейство ограничения върху същите двоични променливи. Комбинацията от двете, многомерност и предхождане едновременно, не е взета от готов източник и начинът, по който е обработена в настоящата разработка, е описан в Раздел IV.

1.5. Изводи от прегледа

Прегледът дава три опорни точки за проектирането. Първо, системата поддържа решение, което остава на човека, затова изходът ѝ трябва да е обяснимо, а не само оптимално множество. Второ, оценяването и подборът са две отделни задачи и могат да се сменят независимо, което е аргумент за ясна граница между двата слоя. Трето, изборът на метод за многокритериална оценка не е безразличен за резултата, затова сравнението между два метода е част от проекта, а не подробност.

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Метод за многокритериална оценка

Разгледани са два метода. Методът на аналитичната йерархия [8] извежда теглата на критериите от попарни сравнения и дава на оценяващия мярка за съгласуваност на собствените му преценки. Цената е обемът на въпросите: за n критерия сравненията са $n(n - 1)/2$, а при добавяне на нов критерий целият набор се преразглежда. Методът TOPSIS [10] изисква теглата да бъдат дадени наготово и класира алтернативите по относителна близост до идеалната точка, което го прави евтин при много алтернативи и стабилен при добавянето на нова.

Изборът е двата метода да бъдат реализирани, а не единият. Причината е емпирична и е посочена в прегледа: върху едни и същи данни различните методи могат да класират различно [11]. Реализирането само на единия би скрило този факт зад числото, което той произвежда. При двата реализирани метода несъвпадението в класирането става наблюдаемо и се превръща в резултат на проекта, а не в мълчаливо допускане. Аналитичната йерархия се използва за извличане на теглата от попарните сравнения, а след това и двата метода се изпълняват върху едни и същи тегла, така че разликата в класирането да идва само от начина на обединяване на критериите, а не от различен вход.

2.2. Метод за решаване на задачата за подбор

Задачата за избор на портфейл при бюджет, капацитет по ресурси и отношения на предхождане е формулирана като задача на целочисленото линейно програмиране. Разгледани са три пътя за решаването ѝ.

1. **Лакома евристика** по съотношение полза към стойност. Евтина за реализация, но не дава гаранция за оптималност и изисква допълнителна обработка на ограниченията за предхождане.
2. **Собствен алгоритъм разклоняване и ограничаване** по схемата на Land и Doig [18] с горна граница от линейната релаксация [15]. Дава точно решение и е напълно прозрачен, но изисква горната граница да бъде изведена и доказана за многомерния случай.
3. **Готов решател** от пакет за целочислено програмиране. Дава точно решение и поема

многомерните ограничения без допълнителна работа, но въвежда външна зависимост, която трябва да бъде налична при построяването.

Избран е вторият път, собствената реализация. Решаващият довод е свързан с разпространението на резултата, а не с алгоритмиката. Хранилището е публично и условието, което проектът си поставя, е чужд човек да може да го клонира и да изпълни две команди без нищо друго освен компилатор за C++20 и CMake. Пакет за целочислено програмиране изисква мениджър на пакети, изтегляне при конфигуриране и съгласуване на версии; построяване, което зависи от тях, на практика не се изпълнява от никого. Същият довод се отнася и за библиотеката за четене на JSON и за рамката за модулно тестване, затова и те са реализирани в хранилището.

Цената на този избор е ясна и се отчита тук, а не се премълчава. Реализацията покрива размерите, които разглежданият случай изисква, десетки проекти, а не хиляди, и не се съревновава с промишлен решател при по-големи задачи. Разделът с резултатите отчита измереното време при нарастващ брой проекти, за да е видимо докъде стига.

Лакомата евристика остава реализирана и се изпълнява винаги, за да служи за долна граница, срещу която се проверява резултатът на точния алгоритъм. Това е и най-евтиният начин грешка в модела да бъде забелязана: точен алгоритъм, който връща по-лош резултат от допустимо решение на евристиката, не е бавен, а сгрешен.

2.3. Технологична основа

Ядрото на модела е реализирано на C++20. Изборът е обоснован от два фактора: анализът на чувствителността изпълнява един и същ модел десетки пъти при изменени входове, а анализът на риска по метода на Монте Карло изпълнява разписанието стотици хиляди пъти, което поставя изисквания към времето за изпълнение. Алтернативата, Python, би съкратила времето за писане, но при броя повторения в Раздел VI разликата в скоростта на изпълнение се превръща в разлика между анализ, който се изпълнява при всяка промяна на сценария, и анализ, който се изпълнява веднъж.

Форматът на сценария е JSON, а не XML. Документът е дърво от числа, низове и списъци без смесено съдържание, без атрибути и без пространства от имена, тоест без нито едно от свойствата, заради които XML е по-изразителен. Съответстващият четец за JSON е няколко пъти по-къс, а разликите между две версии на сценария се четат ред по ред, без затварящите етикети да удвояват шума. Единственото разширение спрямо стандарта

е приемането на коментари с две наклонени черти: сценарият е документ, който човек поддържа, и необяснено число в него е по-лошо от едно допълнително правило в четеща.

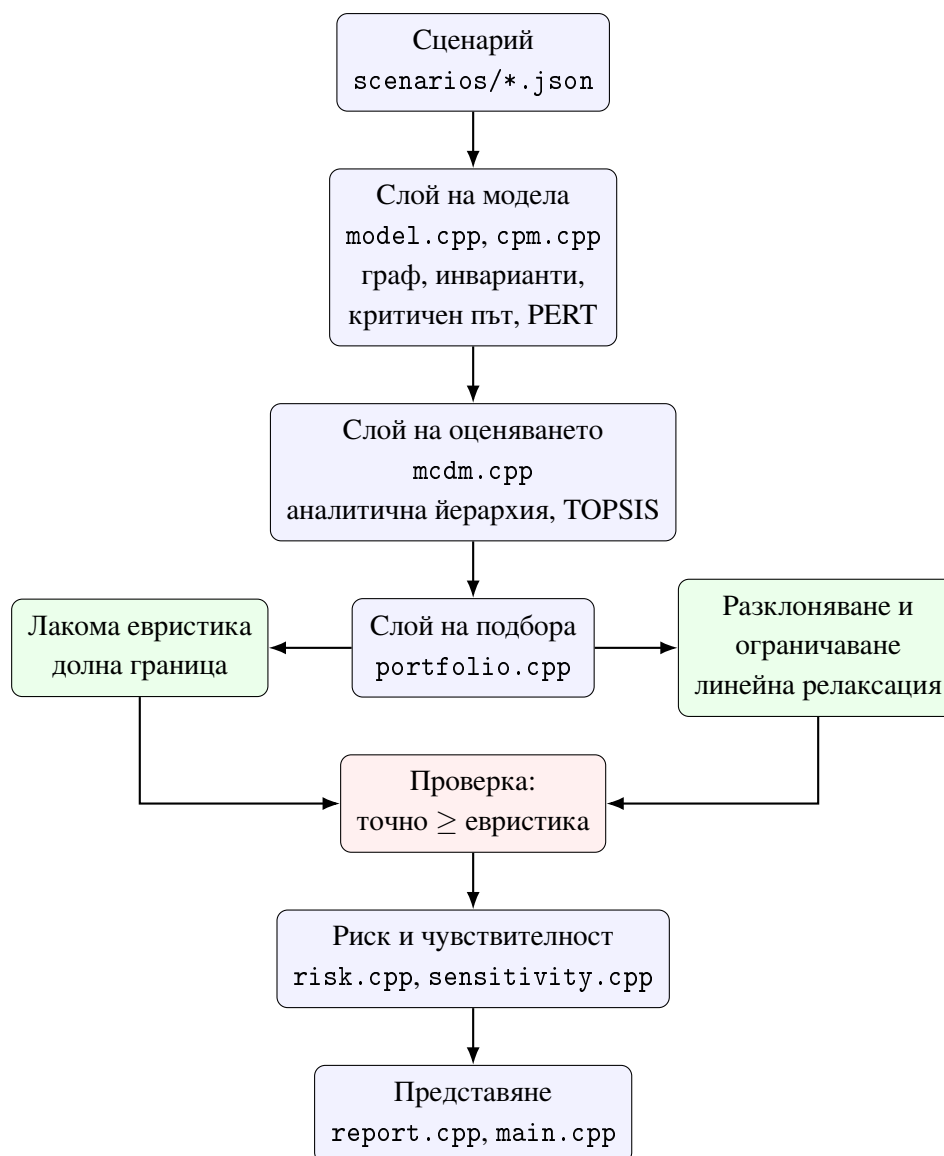
Потребителският интерфейс е команден ред, който извежда текстови таблици. В първоначалния план беше предвиден уеб интерфейс; той отпадна. Резултатът на системата е набор от таблици и обяснения към тях, а не интерактивна работа с данни, така че браузърът би добавил втори език и втора система за построяване без принос към това, което разделът с резултатите показва. Границата между ядрото и представянето остава на същото място, при документа със сценария, така че добавянето на друг интерфейс по-късно не засяга ядрото.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Слоеви и отговорности

Системата е разделена на четири слоя с едноточна зависимост отгоре надолу. Слойт на *модела* съдържа описанието на бизнес процеса и на проектите кандидати и не зависи от нищо друго. Слойт на *оценяването* превръща описанието в число за всеки проект чрез избрания метод за многокритериална оценка. Слойт на *подбора* формулира и решава задачата за избор на портфейл. Слойт на *представянето* приема сценарий, извиква останалите три и връща резултата заедно с обяснението към него.

Границите между слоевете са поставени така, че смяната на метода за оценяване да не засяга подбора, а смяната на алгоритъма за подбор да не засяга оценяването. Това не е абстракция за всеки случай, а пряко следствие от Раздел II: и двата слоя имат по повече от една реализация още в първата версия. Потокът от данни между слоевете е показан на Фиг. 1.



Фигура 1. Слоеви и поток от данни. Стрелките сочат посоката на зависимостта.

Проверката преди последните два блока не е украса на диаграмата. Тя е единственото място, където грешка в построяването на модела може да бъде уловена автоматично: ако точният алгоритъм върне стойност под тази на допустимо решение, намерено от евристиката, изпълнението спира със съобщение за грешка в модела, вместо да отчете число.

3.2. Модел на данните

Бизнес процесът се представя като насочен ацикличен граф. Върховете са дейности с продължителност, стойност и потребен ресурс. Ребрата са отношения на предхождане. Проектът кандидат е свързано подмножество от дейности заедно с оценките си по

критериите. Портфейлът е подмножество от проекти.

Ацикличността е инвариант, който системата проверява при зареждане, а не допускане. Цикъл в графа на предхождането прави най-ранният срок на завършване недефиниран, а задачата за подбор неразрешима; откриването му при зареждането дава на потребителя посочимо място в неговия вход, вместо съобщение за грешка от алгоритъма. Проверката се прави на двете нива поотделно: между дейностите на един проект и между самите проекти.

Таблица 1. Обекти на модела и техните задължителни полета

Обект	Задължителни полета	Инвариант
Дейност	идентификатор, продължителност, стойност	неотрицателни стойности, оптимистична $\leq$ най-вероятна $\leq$ песимистична
Зависимост	източник, цел	графът остава ацикличен, и двата края съществуват
Проект	идентификатор, дейности, оценки по критериите	непразно множество дейности, оценка по всеки критерий
Критерий	идентификатор, посока	сумата на теглата се нормира до единица
Ресурс	идентификатор, капацитет	неотрицателен капацитет
Ограничение	бюджет, капацитет по ресурс, предхождане, взаимно изключване	предхождането и изключването не се отнасят за една и съща двойка

Съставът на модела е обобщен в Табл. 1. Посоката на критерия указва дали по-голямата стойност е по-добра, което е необходимо и за нормализацията в TOPSIS, и за обръщането на колоната при синтеза по аналитичната йерархия.

Три величини не се въвеждат, а се извеждат при зареждането, за да няма два източника на едно и също число. Стойността на проекта е сумата от очакваните стойности на дейностите му. Продължителността на проекта е дължината на неговия критичен път. Потребният ресурс на проекта е върховото едновременно потребление при разписание по най-ранни срокове, изчислено чрез обхождане на моментите на започване и завършване. Последното е съзнателно консервативно: то допуска, че всеки проект работи с върховото

си потребление през целия период, в който е активен, и следователно никога не подценява натоварването.

3.3. Формат на сценария и на резултата

Входът е JSON документ с фиксирана схема. Сценарият съдържа критериите, матрицата на попарните сравнения, ресурсите с техните капацитети, бюджета и проектите с дейностите и оценките им. Пълната схема е дадена в Приложение А.

Изходът е текстов доклад с таблици, а не втори JSON документ. Причината е, че изходът не се подава на друга програма: той се чете от човек и се прилага към решението. За всеки отхвърлен проект се отпечатва причината за отхвърлянето, като причините се търсят в определен ред: непокрито предхождане, взаимно изключване с финансиран проект, недостиг на бюджет, недостиг на ресурс. Ако нито едно от тези не важи, проектът е бил допустим и е отпаднал по стойност, и това се казва изрично, вместо да се посочи ограничение, което не е било определящо.

3.4. Анализ на риска

Тройните оценки в сценария носят несигурност, която очакваната стойност изхвърля. Анализът на риска я връща обратно, като извлича извадка от разпределението на всяка продължителност и всяка стойност и преизчислява разписанието и сумарната стойност за всяка извадка. Резултатът е разпределение на общата стойност на портфейла и на срока на завършване на програмата, от което се отчитат средна стойност, медиана и централен интервал.

Проектното решение тук е разпределението да бъде триъгълно върху трите точки. То се определя изцяло от числата, които сценарият вече съдържа, няма допълнителен параметър за форма, който да бъде предположен, и обратната му функция е в затворен вид, което при стотици хиляди извадки има значение. Разликата спрямо бета разпределението, което стои зад очакваната стойност на PERT, е отчетена в Раздел IV.

3.5. Анализ на чувствителността

Анализът на чувствителността се проектира като повторно изпълнение на същия модел върху систематично изменен вход, а не като отделен алгоритъм. Изменят се теглата

на критериите и бюджетът, всяко в зададен интервал и стъпка. Теглата се изменят по едно наведнъж и след това се нормират наново, така че всяко изпълнение да се отнася до едно-единствено изменено допускане; едновременното изменение би дало резултат, който не може да бъде приписан на определена причина.

Резултатът е множество от портфейли, от които се извежда кои позиции остават избрани при всички изменения и кои се появяват и изчезват. Първите са устойчивата част от препоръката и това е основният изход, предназначен за управленския читател. Освен обобщението по позиции се отпечатва и решетка с по един стълб на изпълнение, защото делът сам по себе си казва колко често една позиция е била финансирана, но не и при какви допускания.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Обща структура на изходния текст

Реализацията следва слоевете от Раздел III. Всеки слой е отделна преводна единица с публичен заглавен файл в `include/quorum/`, а зависимостите между тях са еднопосочни. Външни зависимости няма: библиотеката за четене на JSON, алгоритъмът за оптимизация и рамката за тестване са част от хранилището по причините в Раздел II. Модулите са изброени в Табл. 2.

Таблица 2. Модули на реализацията

Модул	Отговорност	Основни имена
json	четене и записване на JSON документ	parse, dump, Value
model	сценарий, проверка на инвариантите, изведени величини	Scenario, Project, load
cpm	критичен път напред и назад, запас, PERT	schedule, topological_order, peak_demand
mcdm	тегла по аналитична йерархия, класиране по TOPSIS	analyse_comparisons, ahp_scores, topsis_scores
portfolio	построяване и решаване на задачата за подбор	make_instance, solve_exact, solve_greedy
risk	метод на Монте Карло върху избрания портфейл	simulate, triangular
sensitivity	сканиране по тегла и по бюджет	sweep_weights, sweep_budget
report	текстови таблици и обяснения	Table, portfolio, membership

4.2. Слой на модела

Слоят на модела съдържа представянето на графа и проверката на инвариантите му. Проверката за ацикличност се извършва с топологично подреждане при зареждане на сценария. Същото подреждане се преизползва при изчисляването на най-ранните срокове, така че обходът на графа се прави веднъж, а не два пъти. Подреждането е по метода на

Kahn: върховете с нулева входяща степен се изваждат последователно и степените на наследниците им се намаляват.

Съобщението при цикъл е проектно решение, а не подробност. Ако подреждането не обхване всички върхове, върху останалите се пуска обхождане в дълбочина, което намира действителния цикъл и го връща като списък от идентификатори. Класът `src::CycleError` носи този списък, а не само текст, така че извикващият може да го покаже. Разликата за потребителя е между съобщение „графът не е ацикличен“ и съобщение, което назовава дейностите, през които циклите минава.

Едно и също представяне на мрежа обслужва две нива. Вътре в един проект върховете са дейности. Между избраните проекти върховете са цели проекти, а продължителността на всеки е дължината на собствения му критичен път. Функцията `model::programme_network` построява второто ниво и при това отсича ребрата към проекти извън избраното подмножество, защото разписание, което се позовава на нефинансиран проект, е безсмислено.

Върховото потребление на ресурс се изчислява чрез обхождане на моментите на започване и завършване, а не чрез мрежа от периоди. Едновременността може да се промени само в такъв момент, затова обхождането е точно и не изисква избор на стъпка. При еднакво време завършването се обработва преди започването, така че дейност, която приключва в момент t , освобождава ресурса за дейност, която започва в същия момент.

4.3. Слой на оценяването

Двата метода приемат една и съща матрица проекти по критерии и връщат по един вектор от оценки, така че могат да бъдат изпълнени върху идентичен вход и сравнени. Общ абстрактен интерфейс между тях няма: това са две функции с еднакъв подпис и клас с една виртуална операция би бил обвивка без съдържание.

Теглата по аналитичната йерархия се извеждат от главния собствен вектор на матрицата на попарните сравнения чрез степенно повдигане. Матрицата е положителна, затова теоремата на Perron и Frobenius гарантира единствена доминираща собствена стойност и сходимост от произволен положителен начален вектор. Главната собствена стойност $\lambda_{\max}$ се оценява като средно на $(Aw)_i/w_i$, оттам се изчислява индексът на съгласуваност $CI = (\lambda_{\max} - n)/(n - 1)$, а коефициентът на съгласуваност е $CR = CI/RI$, където RI е средният индекс на случайно попълнена реципрочна матрица от същия ред по

таблицата на Saaty [8]. Приетият праг е 0,10, взет от същия източник. При стойност над прага функцията `ahp_weights` не връща тегла, а хвърля `InconsistentJudgementError`: числата, изведени от несъгласувани преценки, са аритметика без смисъл и не бива да продължат надолу по веригата.

Редовете от първи и втори ред се обработват отделно, защото при тях няма тройка преценки, която да си противоречи; за тях CR е нула по построение, а не по изчисление, и делението на $RI = 0$ се избягва.

Проверката на матрицата разграничава два различни проблема. Неквадратна матрица, непожителен елемент или нарушена реципрочност са грешки във входа и водят до `McdmError`. Допускът при реципрочността е два процента, защото човек, който попълва сценария, пише $1/3$ като 0,333, а не като точна дроб; по-строг допуск би отхвърлял коректни файлове.

Синтезът по аналитичната йерархия се прави в разпределителен режим: всяка колона се нормира по собствената си сума и след това се претегля. Колоните, при които по-малката стойност е по-добра, се обръщат преди нормирането. Нула в такава колона би дала безкрайност, затова стойността се ограничава отдолу с най-малката положителна стойност в същата колона, а не с произволна константа.

TOPSIS нормира матрицата по евклидова норма на колоната, претегля я, определя идеалната и най-лошата точка по посоката на всеки критерий и връща относителната близост $d^- / (d^+ + d^-)$. Случаят, в който всички алтернативи съвпадат и двете разстояния са нула, се връща като единица, което е единственият отговор, който не дели на нула.

4.4. Слой на подбора

Задачата се формулира с двоични променливи, по една на проект, целева функция сумата от оценките и ограничения по бюджет, по капацитет за всеки ресурс, по предхождане и по взаимно изключване. Реализирани са два алгоритъма.

Разклоняване и ограничаване. Проектите се обхождат в топологичен ред на отношението на предхождане, тоест всеки проект се решава след предшествениците си. Този ред не е удобство, а източник на две опростявания. При включване на проект предшествениците му вече са решени и е достатъчна една проверка дали всички са включени. При изключване на проект никой вече включен не може да зависи от него, защото зависимите идват по-късно, така че поправка назад не се налага.

Горната граница във всеки връх е стойността на линейната релаксация на остатъка. За едно ограничение релаксацията на задачата за раницата се решава чрез запълване в намаляващ ред на плътността полза към потребление и вземане на дроб от позицията, която пресича границата. Релаксация, при която се запазва само едно от ограниченията, е релаксация и на пълната задача, следователно всяка от тези граници е допустима, а най-малката от тях е също допустима и по-стегната от всяка поотделно. Отношенията на предхождане и на взаимно изключване също отпадат при изчисляването на границата, което само разширява допустимото множество. Наредбите по плътност за всяко измерение се изчисляват веднъж при построяването на търсенето: сортиране във всеки връх би доминирало времето за изпълнение.

Разклоняването започва с включване, а не с изключване. Първото спускане тогава следва наредбата по плътност и достига решение, близко до лакомото, което дава ранна долна граница и позволява отсичането да работи по-надолу в дървото. Търсенето приема и ограничение по брой върхове; при достигането му се връща допустимо, но недоказано оптимално решение, а полето *optimal* остава лъжа, за да не бъде отчетено като точно.

Лаком алгоритъм. Проектите се обхождат в намаляващ ред на съотношението полза към стойност. За всеки кандидат се изчислява затворената обвивка на предшествениците му и цялата обвивка се приема или отхвърля наведнъж. Тук евристиката отстъпва: проект с висока плътност може да повлече евтини по полза предшественици, които точното търсене би отказало.

4.5. Слой на риска

Методът на Монте Карло изпълнява разписанието по веднъж за всеки проект и за всяка извадка. При стотици хиляди извадки построяването на мрежата от идентификатори за всяко теглене би доминирало времето, затова топологичният ред и списъците с предшественици се компилират веднъж в структурата *Compiled*, а в цикъла се изпълнява само проходът напред. Обратният проход не се изпълнява изобщо, защото за срока на завършване запасите не са нужни.

Разпределението е триъгълно и се извлича чрез обратната функция на разпределението, разделена на две в точката на модата. Приетата разлика спрямо бета разпределението зад очакваната стойност на PERT се отчита изрично: средната стойност на триъгълното разпределение е средно аритметично на трите точки, докато

оачкваната стойност на PERT дава на модата тегло четири. При несиметрична оценка двете се различават, и то в предвидима посока, което се вижда в Раздел VI. Генераторът е `std::mt19937_64` с фиксирано начално състояние по подразбиране, така че всяко отчетено число може да бъде възпроизведено чрез повторно изпълнение на същата команда.

4.6. Слой на представянето

Слоят на представянето приема сценарий, изпълнява останалите три слоя и извежда резултата като текстови таблици с фиксирана ширина на стълбовете. Изходът е само от знаци от ASCII: демонстрацията трябва да се чете и на конзола със стара кодова таблица, а знак за рамка, който се изобразява като въпросителна, е по-лош от тире.

Обяснението към препоръката се сглобява тук: за всеки отхвърлен проект се търси първото ограничение, което го е изключило, в реда от Раздел III. Условието за валидност от Раздел V също е реализирано на това ниво, преди отпечатването: ако лакомото решение надхвърли точното, се хвърля изключение и нищо не се отчита.

Командният ред приема пет команди: `demo` изпълнява цялата верига върху вложения сценарий, `solve`, `sensitivity` и `risk` изпълняват по един етап, а `bench` провежда измерванията от Раздел VI.

4.7. Тестове

Проверката на реализацията е разделена на два вида. Модулните тестове покриват отделните изчисления: топологичното подреждане, най-ранните и най-късните срокове върху мрежа с ръчно изчислен отговор, нормализацията в TOPSIS, извеждането на теглата и коефициента на съгласуваност, обратната функция на триъгълното разпределение. Тестовите върху цялата система изпълняват сценарии с известно предварително решение и сравняват изхода с него.

Рамката за тестване е в хранилището, в `tests/check.hpp`: регистър от именувани случаи, няколко макроса за твърдения и филтър по име. CMake регистрира по един запис в CTest за всяка група, като подава името ѝ на един и същ двоичен файл. Така броят на записите в CTest съответства на групите, без да се строят осем отделни изпълними файла.

Два теста носят по-голяма тежест от останалите. Първият сравнява резултата на разклоняването и ограничаването с изчерпателно изброяване на всички подмножества

върху четиридесет малки задачи. Изброяването е написано отделно и не използва нито ред от търсенето, защото независим отговор е независим само ако не споделя код с проверяваното. Вторият проверява, че лакомото решение никога не надхвърля точното, и то върху шестдесет задачи; това е условието, върху което почива целият експеримент.

Общо са реализирани 69 тестови случая в седем групи, плюс изпълнение на демонстрацията от край до край като осми запис в CTest. Инструмент за измерване на покритието не е използван и затова покритие не се отчита: числото щеше да бъде предположение, а не измерване.

V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА

5.1. Какво се проверява

Проверката отговаря на четири въпроса, поставени независимо един от друг. Първо, дава ли системата правилен отговор там, където правилният отговор е известен предварително. Второ, съвпадат ли класиранията на двата метода за многокритериална оценка върху едни и същи данни, и ако не, къде се разминават и достига ли разминаването до избрания портфейл. Трето, каква е цената на точното решаване спрямо лакомата евристика, в качество на решението и във време, при нарастващ размер на задачата. Четвърто, колко устойчива е препоръката спрямо изменение на входните допускания.

Първият въпрос е за коректност и има еднозначен отговор. Останалите три са измервания, чиито резултати не се знаят предварително и не се предполагат тук.

5.2. Среда за измерване

Всички числа в Раздел VI са получени на една и съща машина, в една и съща конфигурация на построяване. Тя е записана в Табл. 3, за да може измерването да бъде повторено.

Таблица 3. Среда, в която са проведени измерванията

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, 6 ядра, 12 логически процесора
Оперативна памет	16 GiB
Операционна система	Windows 11 Pro N, версия 10.0.26200
Компилятор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Стандарт	C++20, -std=c++20
Режим на построяване	CMAKE_BUILD_TYPE=Release, тоест -O3 -DNDEBUG
Система за построяване	CMake 4.3.2 с генератор Ninja 1.13.2
Външни зависимости	няма

Измерването на времето се извършва вътре в програмата с `std::chrono::steady_clock`, около самия алгоритъм, без времето за четене на сценария и за отпечатване. Изпълнението е едно и също нишково; никоя част от

системата не използва повече от една нишка, така че броят логически процесори не влияе на отчетените времена и е записан само за пълнота на описанието.

5.3. Набор от сценарии

Проверката се извършва върху две групи сценарии.

Първата съдържа задачи, породени по зададено правило от генератор с фиксирано начално състояние, с контролиран брой проекти, две измерения на капацитета, отношения на предхождане с вероятност една четвърт на проект и няколко двойки на взаимно изключване. Бюджетът и капацитетите се задават на четиридесет процента от исканото, за да бъде ограничението действащо. Тази група служи за двете количествени измервания: срещу изчерпателно изброяване при малки размери и срещу лакомата евристика при нарастващ размер.

Втората съдържа един съставен сценарий, `scenarios/example.json`, с дванадесет проекта кандидати, четири критерия, два ресурса и бюджет, покриващ около половината от исканото. Той съдържа отношение на предхождане между два проекта, двойка взаимно изключващи се проекти и тройни оценки на всички продължителности и стойности. Върху него се провеждат съпоставката между двата метода за оценяване, анализът на риска и анализът на чувствителността.

Третата група, предвидена в първоначалния план, беше сценарий, съставен по действителна практика на планиране в организация. Такъв не беше получен и групата отпада. Тя не се замества с измислен пример: сценарий, съчинен така, че да изглежда действителен, не носи повече доказателствена сила от породения по правило, а внушава, че носи.

5.4. Процедура

За всеки сценарий се изпълнява следната процедура. Сценарият се зарежда и инвариантите му се проверяват. Теглата на критериите се извличат от матрицата на попарните сравнения и се проверява коефициентът на съгласуваност. Оценките се изчисляват по всеки от двата метода при едни и същи тегла. За всеки вектор от оценки задачата за подбор се решава веднъж с лакомата евристика и веднъж с разклоняване и ограничаване. Резултатите се записват заедно с изразходваното време и броя обходени върхове.

Условие за валидност: резултатът на точния алгоритъм не може да бъде по-лош от този на евристиката. Нарушение на това условие се третира като грешка в модела и спира експеримента, а не се отчита като резултат.

Параметрите на измерванията са следните.

- **Коректност.** Размери 8, 12, 16 и 18 проекта, по двадесет задачи на размер. За всяка задача се сравнява стойността, върната от търсенето, със стойността от изчерпателно изброяване на всички 2^n подмножества.
- **Точно решаване срещу евристика.** Размери от 10 до 38 проекта през четири, по пет задачи на размер. Отчитат се средната стойност на двете решения, отношението помежду им, броят върхове и времената.
- **Сходимост на Монте Карло.** Брой извадки 10^2 , 10^3 , 10^4 , 10^5 и 10^6 върху избрания портфейл на съставния сценарий. Отделно, за да се отдели грешката на извадката от систематичното изместване, при три от тези размери измерването се повтаря с десет различни начални състояния на генератора и се отчита разликата между най-голямата и най-малката получена стойност.
- **Чувствителност по тегла.** Всяко тегло се умножава поотделно по 0,5, 0,75, 1,25 и 1,5 и наборът се нормира наново, което при четири критерия дава шестнадесет изпълнения.
- **Чувствителност по бюджет.** Бюджетът се изменя от шестдесет до сто и четиридесет процента от зададения със стъпка десет процентни пункта, което дава девет изпълнения.

Измерванията се изпълняват с една команда, `quorum bench`, а анализът на чувствителността и рискът с `quorum demo`. И двете команди са част от хранилището и не приемат параметри, които биха променили отчетените числа.

5.5. Форма на резултатите

Резултатите се представят в пет вида. Таблица за съвпадението с изчерпателното изброяване. Таблица на разминаванията между двата метода за многокритериална оценка, с по един ред на всяко разминаване в класирането. Таблица за точното решаване срещу евристиката с по един ред на размер. Таблица за сходимостта на Монте Карло с по един ред на брой извадки. Решетка на устойчивостта, която показва при кой бюджет всяка позиция влиза в препоръчания портфейл и при кой излиза от него.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Коректност върху задачи с известно решение

Таблица 4 отчита съвпадението между стойността, върната от разклоняването и ограничаването, и стойността, получена чрез изчерпателно изброяване на всички подмножества. Числата са от едно изпълнение на quorum bench.

Таблица 4. Съвпадение с изчерпателното изброяване

Проекти	Задачи	Съвпадения	Търсене, ms	Изброяване, ms
8	20	20 от 20	0,0039	0,021
12	20	20 от 20	0,0172	0,365
16	20	20 от 20	0,0432	5,780
18	20	20 от 20	0,1069	32,353

Съвпадението е пълно при всичките осемдесет задачи. Времената в последните два стълба са средни за една задача. Отношението помежду им нараства от около пет пъти при осем проекта до около триста пъти при осемнадесет, което е очакваното поведение: изброяването расте като 2^n независимо от входа, докато търсенето отсича клони и расте съществено по-бавно. Осемнадесет проекта е горната граница на тази проверка не заради търсенето, а заради изброяването: при двадесет и пет проекта еталонното решение вече би отнело часове.

Извън тази таблица коректността се проверява и от 69 модулни теста в седем групи, включително сравнение с изчерпателно изброяване върху четиридесет задачи по дванадесет проекта и проверка върху шестдесет задачи, че лакомото решение никога не надхвърля точното. Всички преминават.

6.2. Съпоставка на двата метода за многокритериална оценка

Теглата на критериите, изведени от матрицата на попарните сравнения в съставния сценарий, са 0,1891 за стратегическото съответствие, 0,3509 за възвръщаемостта, 0,3509 за регулаторната необходимост и 0,1091 за риска. Главната собствена стойност е 4,0098, индексът на съгласуваност 0,0033, а коефициентът на съгласуваност 0,0036, тоест далеч под прага 0,10 и преценките се приемат.

При тези тегла двата метода класират дванадесетте проекта различно на шест

позиции. Разминаванията са изброени в Табл. 5. Коефициентът на рангова корелация на Spearman между двете класирания е 0,9580. В Табл. 5 разликата е мястото по аналитичната йерархия минус мястото по TOPSIS, тоест положителна стойност означава, че TOPSIS класира проекта по-напред.

Таблица 5. Разминавания в класирането между двата метода

Проект	Място, АНР	Място, TOPSIS	Разлика
P09	6	5	+1
P06	8	6	+2
P04	10	9	+1
P10	5	7	-2
P12	7	8	-1
P03	9	10	-1

Очакването от прегледа беше, че разминавания ще има [11], и то се потвърждава. По-съществено обаче е второто наблюдение. Всички шест разминавания са в средата на класирането, между пето и десето място, и нито едно не е с повече от две позиции. Първите четири и последните две места съвпадат.

Отражението върху решението е нулево. Изпълнени поотделно с оценките по аналитичната йерархия и с оценките по TOPSIS, двете изпълнения на подбора връщат един и същ портфейл: P01, P03, P04, P07, P08, P09 и P11. Сумарната оценка, разбира се, е различна, защото двата метода работят в различни мащаби, 0,5694 срещу 3,3411, но множеството съвпада. Изводът е, че при този сценарий изборът между двата метода не е решаващ, и това е по-полезно за управленския читател от която и да е от двете подредби поотделно. Обратното твърдение, че методът никога не е от значение, не следва оттук и не се прави: проверено е върху един сценарий.

6.3. Избраният портфейл и критичният път на програмата

При бюджет 1150 и искане 2106 системата финансира седем от дванадесетте проекта за 1123, тоест 97,7 процента от бюджета. Ограничението по инженерен ресурс е изчерпано напълно, 26,0 от 26,0, докато операционният ресурс остава на 9,0 от 11,0. Търсенето обхожда 357 върха и доказва оптималност за 0,031 ms. Лакомата евристика достига 97,55 процента от оптималната стойност за 0,006 ms.

Критичният път на получената програма минава през P03 и P04 и дава обща

продължителност 34,7 периода. Останалите пет финансирани проекта имат запас между 16,8 и 22,8 периода, тоест забавяне в който и да е от тях не измества края на програмата. Стандартното отклонение по PERT върху критичния път е 2,64 периода, откъдето вероятността програмата да приключи до 37,3 периода е 84,1 процента.

Отхвърлянията идват с причина. И петте отхвърлени проекта се спират от бюджета: остатъкът е 27, а цените им са между 62 и 325. Заслужава отбелязване, че сред тях е P02, класиран първи и по двата метода. Предшественикът му P01 е финансиран, тоест ограничението за предхождане е изпълнено, а взаимно изключване не го засяга; спира го единствено цената. Точно това е случаят, в който подредбата по оценка и съставът на портфейла се разминават, и в който системата дължи на читателя причината, а не само списъка.

6.4. Точно решаване срещу лакома евристика

Таблица 6 отчита двата алгоритъма при нарастващ брой проекти, по пет задачи на размер.

Таблица 6. Разклоняване и ограничаване срещу лакома евристика

Проекти	Точно	Лакомо	Дял, %	Върхове	Точно, ms	Лакомо, ms
10	0,5753	0,5197	89,87	74	0,008	0,007
14	0,7875	0,6940	87,84	225	0,018	0,004
18	1,1170	0,9141	81,11	920	0,077	0,008
22	1,2272	1,0708	87,43	3440	0,237	0,007
26	1,5857	1,2652	79,01	7937	0,753	0,013
30	1,8450	1,6092	86,58	18439	1,710	0,012
34	2,2365	1,8923	84,75	54127	5,530	0,015
38	2,4237	2,1413	88,53	73295	7,862	0,022

Три неща се четат от Табл. 6.

Първо, евристиката губи между десет и двадесет и един процента от оптималната стойност. Загубата не намалява с размера и не показва тенденция: тя се движи между 79,01 и 89,87 процента без ред, което е поведение, характерно за лаком избор при действащи ограничения за предхождане. Затворената обвивка на предшествениците се приема или отхвърля наведнъж, и точно това понякога изтласква позиция, която точното търсене би включило.

Второ, цената на точността е малка в абсолютно изражение. При тридесет и осем проекта търсенето отнема под осем милисекунди. Изпълнението на анализа на чувствителността, който преизчислява подбора шестнадесет пъти, следователно остава под секунда и при задачи, значително по-големи от съставния сценарий. Отношението между двата алгоритъма обаче нараства от около единица при десет проекта до около триста и петдесет при тридесет и осем, тоест изборът на точно решаване има граница и тя е в порядъка на десетки проекти, както беше заявено в Раздел II.

Трето, броят обходени върхове нараства от 74 при десет проекта до 73295 при тридесет и осем, тоест около хиляда пъти при увеличение на размера под четири пъти. Пълното дърво при тридесет и осем проекта има 2^{38} листа, повече от $2,7 \cdot 10^{11}$; обходените 73 хиляди върха са около $3 \cdot 10^{-7}$ от него. Отсичането по линейната релаксация върши почти цялата работа.

Условието за валидност е спазено при всичките четиридесет задачи в таблицата и при шестдесетте задачи в тестовите: лакомото решение никога не надхвърли точното.

6.5. Сходимост на метода на Монте Карло

Таблица 7 отчита оценките при нарастващ брой извадки върху избрания портфейл на съставния сценарий, при фиксирано начално състояние на генератора.

Таблица 7. Оценки на Монте Карло при нарастващ брой извадки

Извадки	Средна стойност	Стойност, p05	Стойност, p95	Срок, p95	Време, ms
10^2	1161,3	1095,3	1219,8	42,465	0,1
10^3	1160,7	1097,0	1227,1	42,711	1,0
10^4	1162,2	1099,2	1226,0	42,949	12,8
10^5	1161,6	1099,3	1226,3	43,029	123,8
10^6	1161,7	1099,6	1226,4	43,010	1422,3

Средната стойност е установена още при сто извадки, до една десета от процента. Опашковите оценки се нуждаят от повече: деветдесет и петият процентил на стойността се мести с 7,3 между сто и хиляда извадки, а след десет хиляди се задържа в интервал под половин единица. Времето расте линейно с броя извадки, както се очаква.

Едно изпълнение обаче не показва колко от разликата между два реда е сходимост и колко е случайност на извадката. Затова измерването е повторено с десет различни начални състояния, при три размера. Резултатът е в Табл. 8.

Таблица 8. Разсейване на оценката за p95 на стойността при десет различни начални състояния

Извадки	Средно от десетте	Размах	Относителен размах, %
10 ³	1226,34	7,99	0,651
10 ⁴	1226,53	2,43	0,198
10 ⁵	1226,50	1,20	0,098

Размахът намалява приблизително с корен от броя извадки, което е очакваното поведение на извадкова оценка. От двете таблици заедно следва работният извод: при десет хиляди извадки оценката за p95 е установена до около два и половина, а това е под една пета от процента при стойност от порядъка 1226, тоест достатъчно за управленско решение. Стойността по подразбиране на командата, двадесет хиляди извадки, е избрана точно оттам, а не по усещане.

6.6. Разпределение на стойността и на срока

При двадесет хиляди извадки върху избрания портфейл общата стойност има средна 1162 и стандартно отклонение 38,6, с деветдесетпроцентов централен интервал [1100; 1227]. Срокът на завършване има средна 37,35 периода, стандартно отклонение 3,32 и деветдесетпроцентов централен интервал [32,02; 43,04] периода.

Два извода от тези числа заслужават внимание.

Първият е разминаването между 1123 и 1162. Детерминираният подбор се извършва върху очаквани стойности по PERT, при които модата получава тегло четири. Извадката се тегли от триъгълно разпределение, чиято средна е средно аритметично на трите точки. Оценките в сценария са несиметрични надясно, песимистичната точка е по-далеч от модата, отколкото оптимистичната, затова триъгълната средна е по-висока. Разликата е 3,5 процента и е следствие от съзнателно избраното разпределение, обосновано в Раздел III, а не от грешка.

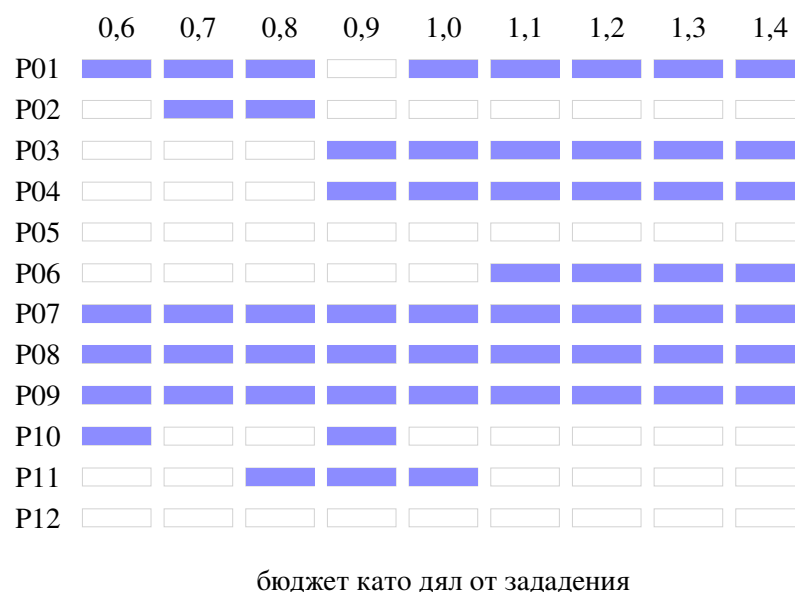
Вторият е по-важен управленски. Портфейлът се побира в бюджета по очаквани стойности, 1123 срещу 1150, но при извадката вероятността действителната сума да остане в бюджета е едва 38,7 процента. Тоест решение, взето само върху очаквани стойности, тук е решение с около три на пет шанс за преразход. Това е точно онова, което една таблица с очаквани стойности не показва, и е основната причина анализът на риска да е част от системата, а не допълнение към нея.

Средният срок от извадката, 37,35 периода, също надхвърля детерминирания 34,7 по същата причина, плюс ефекта на сливането на пътищата: при извадка критичният път не винаги е същият, а максимумът от няколко случайни величини има средна над максимума от средните им.

6.7. Устойчивост на препоръката

При изменение на теглата, шестнадесет изпълнения, пет позиции остават в портфейла при всяко изпълнение: P03, P04, P07, P09 и P11. Две позиции излизат по веднъж, P01 и P08, тоест са избрани в петнадесет от шестнадесет изпълнения. Две позиции влизат само по веднъж, P06 и P10. Три не влизат никога: P02, P05 и P12. Устойчивата част на препоръката спрямо теглата следователно е седем от дванадесетте проекта, пет твърдо и две почти.

При изменение на бюджета, девет изпълнения от шестдесет до сто и четиридесет процента, картината е по-подвижна. Тя е показана на Фиг. 2.



Фигура 2. Позиции в избрания портфейл при изменение на бюджета. Запълнена клетка означава финансиран проект при съответния бюджет.

Три позиции, P07, P08 и P09, са в портфейла при всеки от деветте бюджета. Две, P05 и P12, не влизат при нито един, включително при сто и четиридесет процента: и двете са скъпи и с висок риск, тоест губят по оценка, а не само по бюджет. Между тях са седемте подвижни позиции.

Фигурата показва и нещо, което делът в проценти сам по себе си би скрил.

Присъствието на проект не е монотонно по бюджета. P01 е финансиран при шестдесет, седемдесет и осемдесет процента, отпада при деветдесет и се връща при сто. P11 се появява при осемдесет и изчезва при сто и десет. P10 е финансиран при шестдесет и при деветдесет, но не между тях. Причината е, че при по-широк бюджет става възможна двойката P03 и P04, която измества други позиции; сумарната стойност расте монотонно, но съставът не е монотонен и не може да бъде.

Оттук следва практическото предупреждение. Изречение като „P01 е препоръчан в осемдесет и девет процента от случаите“ е вярно, но подвеждащо, защото внушава интервал, който не съществува. Управленският прочит на препоръката изисква решетката, а не само процента, и затова системата отпечатва и двете.

6.8. Обобщение на измереното

Точното решаване съвпада с изчерпателното изброяване при всичките осемдесет проверени задачи и остава под осем милисекунди до тридесет и осем проекта. Лакомата евристика достига между 79 и 90 процента от оптимума върху породените задачи и 97,55 процента върху съставния сценарий, тоест е добра груба оценка, но не заместител. Двата метода за многокритериална оценка се разминават на шест от дванадесет позиции, но избират един и същ портфейл, тоест разминаването не достига до решението при този сценарий. Анализът на риска показва, че портфейл, който се побира в бюджета по очаквани стойности, има под четиридесет процента вероятност да остане в него, а анализът на чувствителността отделя три безусловно устойчиви позиции от седем подвижни.

ЗАКЛЮЧЕНИЕ

Проектът разработва система за подпомагане на решенията при избор и подреждане на портфейл от ИТ проекти. Бизнес процесът се описва като насочен ацикличен граф от дейности със стойности, продължителности и зависимости. Проектите кандидати се оценяват по няколко критерия чрез два независими метода, а изборът на портфейл се формулира като задача на целочисленото линейно програмиране с ограничения по бюджет, по капацитет и по предхождане. Към препоръката се прилага анализ на чувствителността, който отделя устойчивата ѝ част от частта, която се мени заедно с допусканията.

Предимства на решението. Допусканията са в текстов файл, който се версионира и сравнява, така че препоръката е възпроизводима. Оценяването и подборът са отделени, което позволява методът за оценяване да бъде сменен, без да се пипа подборът. Лакомата евристика, изпълнявана успоредно с решателя, дава евтина проверка, че моделът не е сгрешен.

Недостатъци и ограничения. Системата приема оценките по критериите за дадени и не проверява как са получени; качеството на препоръката не може да надхвърли качеството на този вход. Формулировката е статична и не отчита възможността даден проект да бъде отложен вместо отхвърлен. Обхватът спира при одобряването на портфейла и не покрива проследяването на изпълнението.

Какво показва измерванията. Точното решаване съвпада с изчерпателното изброяване при всичките осемдесет проверени задачи и остава под осем милисекунди до тридесет и осем проекта, което е далеч над размера, който едно планиране на портфейл поставя на практика. Лакомата евристика достига между 79 и 90 процента от оптимума върху породените задачи, тоест е използвана като бърза проверка, но не и като заместител на подбора. Двата метода за многокритериална оценка се разминаха на шест от дванадесет позиции върху съставния сценарий, но избраният портфейл се оказа един и същ; разминаването остана в средата на класирането и не достигна до решението. Най-същественият резултат обаче е от анализа на риска: портфейл, който се побира в бюджета по очаквани стойности, 1123 срещу 1150, има едва 38,7 процента вероятност да остане в него при извадка от разпределенията на входните оценки. Анализът на чувствителността отдели три позиции, устойчиви при всеки разглеждан бюджет, от седем, които се менят, и показва, че съставът на портфейла не е монотонен по бюджета

дори когато сумарната стойност е.

Посоки за по-нататъшна работа. Първата е добавянето на времево измерение, при което решението не е дали един проект да бъде изпълнен, а в кой период да започне. Втората е вграждането на несигурността в оценките, чрез интервали вместо точкови стойности. Третата е свързване с работеща система за планиране на ресурсите на предприятието, така че разчетните стойности и наличният капацитет да се четат оттам, а не да се въвеждат ръчно.

ЛИТЕРАТУРА

- [1] G. A. Gorry and M. S. Scott Morton, “A framework for management information systems,” *Sloan Management Review*, vol. 13, no. 1, pp. 55–70, 1971.
- [2] D. J. Power, *Decision Support Systems: Concepts and Resources for Managers*. Westport, CT: Quorum Books, 2002.
- [3] D. Arnott and G. Pervan, “A critical analysis of decision support systems research,” *Journal of Information Technology*, vol. 20, no. 2, pp. 67–87, 2005.
- [4] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*. Berlin: Springer, 2 ed., 2018.
- [5] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0.2*, 2013. Document formal/2013-12-09.
- [6] J. E. Kelley and M. R. Walker, “Critical-path planning and scheduling,” in *Papers Presented at the December 1-3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference*, pp. 160–173, 1959.
- [7] D. G. Malcolm, J. H. Roseboom, C. E. Clark, and W. Fazar, “Application of a technique for research and development program evaluation,” *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959.
- [8] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York: McGraw-Hill, 1980.
- [9] T. L. Saaty, “How to make a decision: The analytic hierarchy process,” *European Journal of Operational Research*, vol. 48, no. 1, pp. 9–26, 1990.
- [10] C.-L. Hwang and K. Yoon, *Multiple Attribute Decision Making: Methods and Applications*. Berlin: Springer-Verlag, 1981.
- [11] E. Triantaphyllou, *Multi-Criteria Decision Making Methods: A Comparative Study*. Dordrecht: Kluwer Academic Publishers, 2000.
- [12] H. Markowitz, “Portfolio selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

- [13] Project Management Institute, *The Standard for Portfolio Management*. Newtown Square, PA: Project Management Institute, 4 ed., 2017.
- [14] N. P. Archer and F. Ghasemzadeh, “An integrated framework for project portfolio selection,” *International Journal of Project Management*, vol. 17, no. 4, pp. 207–216, 1999.
- [15] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester: John Wiley and Sons, 1990.
- [16] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin: Springer, 2004.
- [17] A. Fréville, “The multidimensional 0-1 knapsack problem: An overview,” *European Journal of Operational Research*, vol. 155, no. 1, pp. 1–21, 2004.
- [18] A. H. Land and A. G. Doig, “An automatic method of solving discrete programming problems,” *Econometrica*, vol. 28, no. 3, pp. 497–520, 1960.

ПРИЛОЖЕНИЯ

Приложение А. Схема на сценария

Сценарият е един JSON документ. Полетата му са изброени в Табл. 9. Всяка продължителност и всяка стойност се задава или с едно число, или с тройна оценка; едното число е случаят, в който трите точки съвпадат, затова за него няма отделен път в кода.

Таблица 9. Полета на входния документ

Поле	Тип	Задължително	Ограничение
name	низ	не	свободен текст
budget	число	да	неотрицателно
criteria	списък	да	непразен
criteria[].id	низ	да	уникален
criteria[].direction	низ	не	max или min, по подразбиране max
criteria[].weight	число	условно	неотрицателно; задължително, ако липсва comparisons
comparisons	матрица	условно	квадратна върху критериите, положителна, реципрочна
resources	списък	не	може да е празен
resources[].id	низ	да	уникален
resources[].capacity	число	да	неотрицателно
projects	списък	да	непразен
projects[].id	низ	да	уникален
projects[].scores	обект	да	по една оценка за всеки критерий
projects[].requires	списък	не	съществуващи идентификатори, без цикъл
projects[].excludes	списък	не	съществуващи идентификатори, различни от requires
projects[].activities	списък	да	непразен
...activities[].id	низ	да	уникален в рамките на проекта
...duration	число	условно	неотрицателно; алтернатива на тройката
...optimistic, likely, pessimistic	числа	условно	$o \leq m \leq p$, неотрицателни
...cost	число	условно	неотрицателно; алтернатива на тройката
...cost_optimistic, cost_likely, cost_pessimistic	числа	условно	$o \leq m \leq p$, неотрицателни

продължение на Табл. 9

Поле	Тип	Задължително	Ограничение
...depends_on	списък	не	съществуващи дейности, без цикъл
...demand	обект	не	по ресурс, неотрицателно

Четецът приема и коментари от две наклонени черти до края на реда, което не е част от стандарта на JSON. Обосновката е в Раздел II. Началото на вложения сценарий е показано в Листинг 1.

```

1 // Bundled scenario: one planning round of an IT department with twelve
2 // candidate projects and roughly a third of the money they ask for.
3 //
4 // Durations are in weeks, costs in thousands of the reporting currency,
  ↪ demand
5 // in engineers held for the length of the activity. Every duration and cost
  ↪ is a
6 // three-point estimate, so the same document drives both the deterministic
7 // schedule and the Monte Carlo pass.
8 {
9   "name": "IT portfolio, planning round",
10  "budget": 1150,
11
12  "criteria": [
13    { "id": "strategic", "direction": "max", "label": "fit with the strategy"
  ↪ },
14    { "id": "roi",          "direction": "max", "label": "expected return" },
15    { "id": "compliance", "direction": "max", "label": "regulatory necessity"
  ↪ },
16    { "id": "risk",        "direction": "min", "label": "delivery risk" }
17  ],
18
19  // Pairwise judgements over the criteria, on Saaty's scale. Read row
  ↪ against
20  // column: entry (i,j) is how much more important criterion i is than j.
21  "comparisons": [
22    [1,    0.5,   0.5,   2],
23    [2,    1,     1,     3],
24    [2,    1,     1,     3],

```

```

25     [0.5,    0.333, 0.333, 1]
26 ],
27
28 "resources": [
29     { "id": "dev", "label": "engineers available in parallel", "capacity": 26
↪ },
30     { "id": "ops", "label": "operations staff available in parallel", "
↪ capacity": 11 }
31 ],
32
33 "projects": [
34     {
35         "id": "P01", "name": "Identity platform migration",
36         "scores": { "strategic": 8, "roi": 4, "compliance": 7, "risk": 6 },
37         "activities": [
38             { "id": "a1", "name": "directory inventory", "optimistic": 2, "likely
↪ ": 3, "pessimistic": 5,
39             "cost_optimistic": 18, "cost_likely": 22, "cost_pessimistic": 30, "
↪ demand": { "dev": 2 } },
40             { "id": "a2", "name": "platform build", "optimistic": 5, "likely": 7,
↪ "pessimistic": 12,
41             "cost_optimistic": 70, "cost_likely": 85, "cost_pessimistic": 120,
42             "depends_on": ["a1"], "demand": { "dev": 4, "ops": 1 } },

```

Листинг 1: Начало на scenarios/example.json

Приложение Б. Изход на системата

Интерфейсът е команден ред и изходът е текст, затова екранни форми в смисъла на графичен интерфейс няма и това приложение ги замества. Листинг 2 съдържа целия изход на командата `quorum demo` върху вложения сценарий, включително обяснението за всеки отхвърлен проект и решетката на устойчивостта. Това е изпълнението, от което са взети числата в Раздел VI.

```

Quorum: portfolio selection under budget and capacity limits
Reading scenarios/example.json

```

```

Scenario
=====

```

Scenario : IT portfolio, planning round
 Candidates : 12 projects, 4 criteria
 Budget : 1150
 Asked for : 2106 (54.6 per cent of the ask is fundable)
 Capacity : dev = 26.0 (engineers available in parallel)
 Capacity : ops = 11.0 (operations staff available in parallel)

Criteria weights from the pairwise judgements

```
=====
criterion  direction  weight  label
-----
strategic   max         0.1891  fit with the strategy
roi         max         0.3509  expected return
compliance  max         0.3509  regulatory necessity
risk        min         0.1091  delivery risk
```

principal eigenvalue 4.0098, consistency index 0.0033, consistency ratio 0.0036
 judgement matrix accepted against the 0.10 threshold

Rankings, AHP against TOPSIS

```
=====
project  AHP score  rank  TOPSIS score  rank  shift
-----
P02      0.0966    1    0.6101       1    .
P08      0.0946    2    0.5947       2    .
P01      0.0915    3    0.5722       3    .
P05      0.0900    4    0.5654       4    .
P10      0.0824    5    0.4903       7   -2
P09      0.0813    6    0.5226       5   +1
P12      0.0811    7    0.4671       8   -1
P06      0.0805    8    0.5045       6   +2
P03      0.0797    9    0.4277      10   -1
P04      0.0796   10    0.4385       9   +1
P07      0.0756   11    0.3936      11    .
P11      0.0671   12    0.3919      12    .
```

Spearman rank correlation between the two rankings: 0.9580
 projects placed differently by the two methods: 6 of 12

Selected portfolio, scored by TOPSIS

```
=====
funded  cost  duration  value  requires
-----
P01     158   15.2    0.5722  -
P03     227   22.0    0.4277  -
P04     114   12.7    0.4385  P03
P07     110   11.8    0.3936  -
P08     141   14.2    0.5947  -
P09     193   17.8    0.5226  -
P11     180   17.8    0.3919  -
```

total value 3.3411, cost 1123 of 1150 budget, dev 26.0 of 26.0, ops 9.0 of 11.0
 branch and bound: proved optimal, 357 nodes, 0.031 ms
 greedy heuristic: value 3.2592 (97.55 per cent of the optimum), 0.006 ms

Not funded, and why:

```
project  cost  value  reason
-----
P02      110   0.6101  costs 110 against 27 of remaining budget
P05      325   0.5654  costs 325 against 27 of remaining budget
P06      213   0.5045  costs 213 against 27 of remaining budget
P10       62   0.4903  costs 62 against 27 of remaining budget
P12      273   0.4671  costs 273 against 27 of remaining budget
```

Programme schedule of the funded portfolio

=====

project	duration	early start	early finish	late start	float	critical
-----	-----	-----	-----	-----	-----	-----
P01	15.2	0.0	15.2	19.5	19.5	
P03	22.0	0.0	22.0	0.0	0.0	yes
P04	12.7	22.0	34.7	22.0	0.0	yes
P07	11.8	0.0	11.8	22.8	22.8	
P08	14.2	0.0	14.2	20.5	20.5	
P09	17.8	0.0	17.8	16.8	16.8	
P11	17.8	0.0	17.8	16.8	16.8	

programme duration 34.7 periods

critical path : P03 -> P04

PERT deviation : 2.64 periods, so a 84.1 per cent chance of finishing within 37.3

Monte Carlo over the funded portfolio

=====

quantity	mean	std dev	p05	p50	p95
-----	-----	-----	-----	-----	-----
total cost	1162	38.6	1100	1161	1227
completion	37.35	3.32	32.02	37.27	43.04

20000 samples in 16.6 ms

90 per cent central interval on cost : [1100, 1227]

90 per cent central interval on completion: [32.02, 43.04] periods

probability the portfolio stays within the budget of 1150: 38.7 per cent

Sensitivity to the criteria weights

=====

one criterion weight varied at a time, 16 runs in 0.5 ms

project	selected in	share	verdict
-----	-----	-----	-----
P01	15 of 16	93.8 %	knife-edge
P02	0 of 16	0.0 %	never
P03	16 of 16	100.0 %	stable
P04	16 of 16	100.0 %	stable
P05	0 of 16	0.0 %	never
P06	1 of 16	6.2 %	knife-edge
P07	16 of 16	100.0 %	stable
P08	15 of 16	93.8 %	knife-edge
P09	16 of 16	100.0 %	stable
P10	1 of 16	6.2 %	knife-edge
P11	16 of 16	100.0 %	stable
P12	0 of 16	0.0 %	never

Sensitivity to the budget

=====

budget varied from 60 to 140 per cent, 9 runs in 0.3 ms

project	selected in	share	verdict
-----	-----	-----	-----
P01	8 of 9	88.9 %	knife-edge
P02	2 of 9	22.2 %	knife-edge
P03	6 of 9	66.7 %	knife-edge
P04	6 of 9	66.7 %	knife-edge
P05	0 of 9	0.0 %	never
P06	4 of 9	44.4 %	knife-edge
P07	9 of 9	100.0 %	stable
P08	9 of 9	100.0 %	stable

P09	9 of 9	100.0 %	stable
P10	2 of 9	22.2 %	knife-edge
P11	3 of 9	33.3 %	knife-edge
P12	0 of 9	0.0 %	never

Funded under each budget:

project	budget x0.60	budget x0.70	budget x0.80	budget x0.90	budget x1.00	budget x1.10	budget x1.20	budget x1.30	budget
↪ x1.40									
↪ -----									
P01	x	x	x	.	x	x	x	x	x
P02	.	x	x	.	.	.	.	.	.
P03	.	.	.	x	x	x	x	x	x
P04	.	.	.	x	x	x	x	x	x
P05	.	.	.	.	.	.	.	.	.
P06	.	.	.	.	.	x	x	x	x
P07	x	x	x	x	x	x	x	x	x
P08	x	x	x	x	x	x	x	x	x
P09	x	x	x	x	x	x	x	x	x
P10	x	.	.	x	.	.	.	.	.
P11	.	.	x	x	x	.	.	.	.
P12	.	.	.	.	.	.	.	.	.

Done.

Листинг 2: Изход на quorum demo

Приложение В. Изходен текст на ключовите изчисления

Листингите по-долу се вмъкват от действителните файлове по маркери в изходния текст, а не се преписват, така че не могат да се разминат с това, което се компилира.

```

101 std::vector<std::size_t> topological_order(const Network& net) {
102     const auto map = index_map(net);
103     const std::size_t n = net.tasks.size();
104     std::vector<std::vector<std::size_t>> succ(n);
105     std::vector<std::size_t> indegree(n, 0);
106
107     for (std::size_t i = 0; i < n; ++i) {
108         for (const auto& dep : net.tasks[i].depends_on) {
109             auto it = map.find(dep);
110             if (it == map.end())
111                 throw UnknownTaskError("activity '" + net.tasks[i].id + "'
↪ depends on unknown '" +
112                                     dep + "'");
113             succ[it->second].push_back(i);
114             ++indegree[i];
115         }

```

```

116     }
117
118     std::vector<std::size_t> ready;
119     for (std::size_t i = 0; i < n; ++i)
120         if (indegree[i] == 0) ready.push_back(i);
121
122     std::vector<std::size_t> order;
123     order.reserve(n);
124     while (!ready.empty()) {
125         std::size_t v = ready.back();
126         ready.pop_back();
127         order.push_back(v);
128         for (std::size_t w : succ[v])
129             if (--indegree[w] == 0) ready.push_back(w);
130     }
131
132     if (order.size() != n) {
133         std::vector<char> alive(n, 0);
134         for (std::size_t i = 0; i < n; ++i) alive[i] = indegree[i] > 0 ? 1 :
↪ 0;
135         throw CycleError(extract_cycle(net, succ, alive));
136     }
137     return order;
138 }

```

Листинг 3: Топологично подреждане с посочване на цикъла, src/crm.cpp

```

75 AhpResult analyse_comparisons(const Matrix& m) {
76     check_square_positive(m);
77     const std::size_t n = m.size();
78
79     // Principal eigenvector by power iteration. The matrix is positive, so
80     // Perron-Frobenius guarantees a single dominant eigenvalue and
↪ convergence
81     // from any positive start vector.
82     std::vector<double> w(n, 1.0 / static_cast<double>(n));
83     for (int iter = 0; iter < kPowerIterations; ++iter) {
84         std::vector<double> next(n, 0.0);

```

```

85     for (std::size_t i = 0; i < n; ++i)
86         for (std::size_t j = 0; j < n; ++j) next[i] += m[i][j] * w[j];
87     next = normalised(std::move(next));
88     double delta = 0.0;
89     for (std::size_t i = 0; i < n; ++i) delta = std::max(delta, std::fabs
↪ (next[i] - w[i]));
90     w.swap(next);
91     if (delta < kConverged) break;
92 }
93
94 AhpResult out;
95 out.weights = w;
96
97 // lambda_max as the mean of (A w)_i / w_i, the standard estimate.
98 double lambda = 0.0;
99 for (std::size_t i = 0; i < n; ++i) {
100     double row = 0.0;
101     for (std::size_t j = 0; j < n; ++j) row += m[i][j] * w[j];
102     lambda += row / w[i];
103 }
104 out.lambda_max = lambda / static_cast<double>(n);
105
106 if (n > 2) {
107     out.consistency_index =
108         (out.lambda_max - static_cast<double>(n)) / static_cast<double>(n
↪ - 1);
109     const double ri = random_index(n);
110     out.consistency_ratio = ri > 0.0 ? out.consistency_index / ri : 0.0;
111 } else {
112     // Orders one and two are consistent by construction: there is no
↪ triple
113     // of judgements that could contradict itself.
114     out.consistency_index = 0.0;
115     out.consistency_ratio = 0.0;
116 }
117 out.consistent = out.consistency_ratio <= kConsistencyThreshold;
118 return out;
119 }

```

Листинг 4: Тегла и коефициент на съгласуваност по аналитичната йерархия, `src/mcdm.cpp`

```
120 double relaxation_bound(std::size_t from, double budget_left,
121                         const std::vector<double>& capacity_left) const {
122     double best = std::numeric_limits<double>::infinity();
123     const std::size_t dims = 1 + inst_.capacity.size();
124     for (std::size_t d = 0; d < dims; ++d) {
125         const std::vector<double>& use = dimension(d);
126         double room = d == 0 ? budget_left : capacity_left[d - 1];
127         double total = 0.0;
128         for (std::size_t i : density_order_[d]) {
129             if (position_[i] < from) continue; // already decided
130             if (use[i] <= 0.0) {
131                 total += inst_.value[i];
132                 continue;
133             }
134             if (use[i] <= room) {
135                 room -= use[i];
136                 total += inst_.value[i];
137             } else {
138                 total += inst_.value[i] * room / use[i];
139                 break;
140             }
141         }
142         best = std::min(best, total);
143     }
144     return best;
145 }
146
```

Листинг 5: Горна граница от линейната релаксация, `src/portfolio.cpp`

Приложение Г. Пълни резултати от измерванията

Листинг 6 съдържа целия изход на командата `quorum bench`, от която са извлечени Табл. 4, Табл. 6, Табл. 7 и Табл. 8. Средата на измерването е описана в Табл. 3.

Quorum benchmarks. Every number below was produced by this run.

Branch and bound against exhaustive enumeration

=====

twenty instances per size, every subset enumerated for the reference answer

projects	instances	matched	search ms	enumeration ms	speed-up
8	20	20/20	0.0039	0.021	5.4x
12	20	20/20	0.0172	0.365	21.2x
16	20	20/20	0.0432	5.780	133.9x
18	20	20/20	0.1069	32.353	302.8x

Branch and bound against the greedy heuristic

=====

five instances per size, two capacity dimensions, budget at 40 per cent of the ask

projects	exact value	greedy value	greedy of optimum	nodes	exact ms	greedy ms	proved
10	0.5753	0.5197	89.87 %	74	0.008	0.007	5/5
14	0.7875	0.6940	87.84 %	225	0.018	0.004	5/5
18	1.1170	0.9141	81.11 %	920	0.077	0.008	5/5
22	1.2272	1.0708	87.43 %	3440	0.237	0.007	5/5
26	1.5857	1.2652	79.01 %	7937	0.753	0.013	5/5
30	1.8450	1.6092	86.58 %	18439	1.710	0.012	5/5
34	2.2365	1.8923	84.75 %	54127	5.530	0.015	5/5
38	2.4237	2.1413	88.53 %	73295	7.862	0.022	5/5

Monte Carlo convergence

=====

scenario IT portfolio, planning round, 7 funded projects, fixed seed

samples	mean cost	p05 cost	p95 cost	p95 completion	ms
100	1161.3	1095.3	1219.8	42.465	0.1
1000	1160.7	1097.0	1227.1	42.711	1.0
10000	1162.2	1099.2	1226.0	42.949	12.8
100000	1161.6	1099.3	1226.3	43.029	123.8
1000000	1161.7	1099.6	1226.4	43.010	1422.3

Sampling error across seeds

=====

samples	p95 cost, ten seeds	spread	relative spread
-----	-----	-----	-----

1000	1226.34	7.99	0.651 %
10000	1226.53	2.43	0.198 %
100000	1226.50	1.20	0.098 %

Листинг 6: Изход на quorum bench